SIMATS ENGINEERING

ASSESSMENT TOOL 3 – REVERSE ENGINEERING

Course Name: Artificial Intelligence Course Code: CSA17

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Student Name: Yuvasri.S

Register Number: 192511202

Duration: 1 Hour **Total Marks:** 50

CODE OF CONDUCT

I Yuvasri.S (192511202) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

QUESTION 1 – MACHINE LEARNING TECHNIQUES FOR REVERSE ENGINEERING

1. Problem Understanding

The problem is to determine whether machine-learning methods can identify similarities, programming patterns and software-component categories from source-code samples. Manual comparison becomes difficult when the number of programs is large. The system should therefore convert source code into measurable features and use ML to classify or cluster programs.

2. Student Activity / Engineering Approach

Collect a safe dataset of non-executable source-code samples from open-source repositories. Label samples by programming language, component type or known software category. Extract lexical, structural and statistical features, then divide the dataset into training, validation and test sets. Apply more than one ML algorithm and compare their performance.

3. Proposed Solution

The proposed pipeline is: source-code dataset → preprocessing → feature extraction → feature vectors → ML models → classification/clustering → similarity analysis → evaluation. Useful features include token frequencies, keywords, operators, function counts, class counts, control-flow indicators, comment ratio, identifier patterns and dependency counts. For classification, Decision Tree, Random Forest, SVM and Naive Bayes can be compared. For unsupervised analysis, K-Means or hierarchical clustering can group similar programs.

4. Methodology

First remove comments or normalize them when they are not relevant to the experiment. Tokenize code and calculate lexical features. Structural features can be extracted from an abstract syntax tree. Normalize numerical features and encode categorical information. Train supervised models using labelled samples. For clustering, select a similarity metric and evaluate cluster quality. Finally compare accuracy, precision, recall, F1-score and confusion matrices for classification, and silhouette score for clustering.

5. Tools and Data

Python can be used with scikit-learn for ML, pandas for dataset handling and tree-sitter or language-specific parsers for safe source-code parsing. Visualizations can show feature importance, confusion matrices and cluster distributions. Only source code intended for analysis should be used; the experiment does not require executing unknown programs.

6. Analysis and Evaluation

Tree-based models are useful because feature importance can show which code characteristics contribute to classification. SVM can work well in high-dimensional feature spaces, while Naive Bayes provides a simple probabilistic baseline. A major limitation is that source-code style differs between developers, so a model may learn superficial coding style instead of genuine program functionality. Cross-project testing is therefore important.

7. Expected Outcome

The experiment demonstrates which ML technique is most suitable for a particular reverse-engineering task. A strong result would be a model that generalizes to previously unseen projects rather than only memorizing samples from the training repositories.

8. Ethical and Professional Considerations

Use only safe, non-executable source-code datasets. Respect repository licenses and avoid collecting private or proprietary code. Results should be reported honestly, including failed experiments and limitations.

QUESTION 2 – AI FOR SOURCE CODE ANALYSIS AND PROGRAM UNDERSTANDING

1. Problem Understanding

The problem is understanding an unfamiliar software repository efficiently. Large repositories contain many functions, classes, variables, dependencies and configuration files, making manual comprehension time-consuming. AI can assist by extracting structure and generating natural-language explanations, but generated explanations must be verified against the

actual code.

2. Student Activity / Engineering Approach

Select an open-source repository suitable for study. Build a map of files, functions, classes and dependencies. Use an AI-assisted analysis method to explain selected functions and modules. Compare the generated explanations with the actual implementation and record correct statements, missing details and incorrect interpretations.

3. Proposed Solution

The proposed architecture is: source repository → parser/static analyzer → program structure → AI analysis → natural-language explanation → human verification. Static analysis identifies symbols and relationships before the AI model receives relevant code context. The AI can summarize function purpose, inputs, outputs, dependencies and important control-flow behaviour.

4. Methodology

Start by indexing the repository. Extract function names, parameters, return values, classes, imports and call relationships. Build a dependency graph. Select representative functions and provide their actual code plus relevant context to the AI system. Ask for structured explanations. A reviewer then checks every major claim against the code. Accuracy can be calculated as the percentage of verified statements that are correct.

5. Tools and Data

Useful tools include Python AST for Python repositories, tree-sitter for multi-language parsing, dependency-graph tools, Git for repository history and an AI/LLM assistant for explanation generation. Visuals can include repository structure diagrams and call/dependency graphs.

6. Analysis and Evaluation

AI explanations are useful for documentation and onboarding, but they may hallucinate behaviour, ignore edge cases or misunderstand dynamic features. Performance should therefore be evaluated using correctness, completeness and consistency. A useful experiment is to compare AI-only explanation with parser-assisted AI explanation. The parser-assisted approach should reduce unsupported claims because the model receives explicit structural information.

7. Expected Outcome

The study should show that AI can reduce the time needed to understand unfamiliar code while demonstrating that human verification remains necessary for high-confidence technical documentation.

8. Ethical and Professional Considerations

Do not upload confidential source code to external AI services without authorization. Respect licenses, protect credentials and secrets, and clearly identify AI-generated documentation as requiring verification.

QUESTION 3 – AI-BASED MALWARE REVERSE ENGINEERING AND ANALYSIS

1. Problem Understanding

The problem is to support malware researchers by automatically identifying suspicious software patterns. The study should remain safe and focus on non-executable datasets or published research samples. AI can classify files using static characteristics such as imported APIs, strings, headers and byte-level features.

2. Student Activity / Engineering Approach

Use a publicly available research dataset containing labelled benign and malicious samples or pre-extracted static features. Do not execute samples. Prepare the features, train classification models and evaluate whether the models can distinguish suspicious from benign patterns.

3. Proposed Solution

The safe architecture is: research dataset → static-feature preprocessing → feature selection → ML classifier → risk prediction → evaluation. Features can include PE-header information, imported API categories, section statistics, printable-string characteristics and entropy-related measurements when already provided by the dataset.

4. Methodology

Clean missing values, encode categorical features, scale where required and divide the dataset into training and testing sets. Compare models such as Random Forest, SVM and Logistic Regression. Evaluate accuracy together with precision, recall, F1-score and ROC-AUC. Because false negatives can be serious in security, recall for the malicious class should receive particular attention.

5. Tools and Data

Python, pandas and scikit-learn are appropriate for the experiment. Safe public feature datasets can be stored as CSV files. Confusion matrices, ROC curves and feature-importance plots provide useful visuals. No malware execution, unpacking or operational deployment is required.

6. Analysis and Evaluation

A high accuracy score alone can be misleading if the dataset is imbalanced or contains duplicate/near-duplicate samples. A stronger evaluation removes duplicates, uses stratified splits and tests on a separate source or time period. Feature importance can help identify which static characteristics influence predictions. The main limitation is concept drift: malware families change, so a model trained on older samples may perform poorly on newer families.

7. Expected Outcome

The experiment demonstrates how AI can support malware triage and prioritization. A practical system could assign a risk score for further human investigation rather than automatically declaring a file malicious.

8. Ethical and Professional Considerations

Malware research must be conducted only in authorized, isolated environments and with safe datasets. The proposed activity intentionally avoids execution and operational malware-development guidance. Researchers should protect sensitive samples and report findings responsibly.

QUESTION 4 – REVERSE ENGINEERING OF APIs USING AI

1. Problem Understanding

The problem is to recover API behaviour when documentation is incomplete. An API can be understood by examining controlled request-response examples, source code where available and observed data formats. The goal is to infer endpoints, parameters, authentication requirements, response schemas and relationships without attacking unauthorized systems.

2. Student Activity / Engineering Approach

Select a documented/open API or a locally controlled test API. Record valid requests and responses. Use an AI-assisted method to infer endpoint purpose, parameters, request formats and response structures. Generate documentation and compare it against the official specification.

3. Proposed Solution

The architecture is: controlled API → request/response collection → schema extraction → AI interpretation → generated documentation → specification comparison. Structured observations provide evidence, while AI converts those observations into human-readable descriptions and examples.

4. Methodology

Create a set of legitimate test cases covering normal inputs, optional parameters and expected responses. Extract HTTP method, endpoint path, status code, request parameters and response fields. Provide these observations to the AI. Generate documentation containing endpoint purpose, parameters, example requests, response fields and error cases. Compare generated documentation with the known API specification and calculate coverage and correctness.

5. Tools and Data

Use Python requests or a similar client only against the authorized test API, OpenAPI/Swagger specifications, JSON schema tools and an AI assistant. Tables can compare actual and inferred fields, while diagrams can show endpoint relationships.

6. Analysis and Evaluation

Evaluate endpoint coverage, parameter accuracy, response-schema accuracy and incorrect assumptions. AI may infer a field's purpose incorrectly when the available observations are limited. It may also miss authentication or error-handling details. More diverse controlled test cases improve confidence. The strongest approach combines observed evidence with the official specification instead of treating AI output as authoritative.

7. Expected Outcome

The expected outcome is automatically generated, understandable API documentation that closely matches the real interface. The exercise demonstrates how AI can accelerate documentation recovery while requiring verification.

8. Ethical and Professional Considerations

Testing must be restricted to APIs for which permission has been given. Do not probe third-party services, bypass authentication, evade rate limits or attempt to access hidden data. Credentials and personal data should never be included in the dataset.

OVERALL COMPARISON AND EVALUATION

Question 1: ML techniques classify and cluster source-code characteristics. The key evaluation is generalization across projects and programming styles.

Question 2: AI assists program understanding and documentation. The key evaluation is factual correctness and completeness of generated explanations.

Question 3: ML supports safe malware triage from static features. The key evaluation is recall, F1-score, ROC-AUC and robustness to changing malware patterns.

Question 4: AI assists controlled API behaviour and documentation recovery. The key evaluation is endpoint, parameter and schema accuracy.

CONCLUSION

Reverse engineering is increasingly supported by AI techniques that can classify code, explain unfamiliar programs, identify suspicious software characteristics and recover API documentation. Machine learning provides scalable pattern recognition, while AI-assisted analysis can reduce the time required for program understanding. However, the experiments also show that AI outputs must be evaluated against reliable evidence. Dataset quality, project diversity, class imbalance, changing software behaviour and hallucinated explanations can affect results. Safe datasets, controlled environments, human verification and ethical use are therefore essential. The four designs collectively demonstrate technical accuracy, engineering problem identification, effective use of tools, clear communication and professional responsibility, satisfying the major requirements of the assessment.

SUBMISSION DETAILS

Name: Yuvasri.S

Register Number: 192511202

Course: Artificial Intelligence – CSA17

Assessment Tool: 3 – Reverse Engineering